

AGROFLOW – Planning 36h + Prompts Cursor optimisés

Version compressée du sprint 48h, adaptée à une fenêtre de 36h. Même logique : ne pas construire toute la plateforme, livrer un parcours complet et impeccable.

1. Vue d'ensemble du planning 36h

Créneau	Durée	Objectif
H0 – H2	2h	Alignement + setup projet
H2 – H8	6h	Cœur du MVP (data + compatibilité + regroupement)
H8	checkpoint	Démo interne minimale
H8 – H16	8h	Parcours complet (dashboard, matching, grouping, opération, passeport)

Créneau	Durée	Objectif
H16	checkpoint	MVP fonctionnel de bout en bout
H16 – H22	6h	Polish + effets "wow"
H22 – H26	4h	Stabilisation / bug fixing
H26 – H29	3h	Préparation du pitch
H29 – H32	3h	Répétitions
H32 – H36	4h	Sécurisation finale + marge tampon

La logique reste la pyramide de priorité du document initial :

- **Niveau 1 (indispensable)** : création de lot, matching, regroupement.
- **Niveau 2 (important)** : opération, timeline, dashboard.
- **Niveau 3 (wow)** : QR code, carte, animations, alertes.

En 36h, si le temps manque, on coupe d'abord dans le Niveau 3, jamais dans le Niveau 1.

2. Détail par phase

PHASE 1 — H0 à H2 : Alignement + Setup

H0 (30 min) — Kickoff

- Stack validée : React + Vite + Tailwind + LocalStorage/JSON (pas de backend distant).
- Librairies : `react-router-dom`, `lucide-react`, `qrcode.react`, `recharts`.
- Scénario de démo figé dès maintenant (3 producteurs, même produit, destination commune) — ne plus y toucher après.

H0:30 – H1:30 — Initialisation technique

- Sergio : `npm create vite@latest agroflow`, structure de dossiers (`components/`, `pages/`, `data/`, `services/`, `utils/`, `types/`).
- Répartition des rôles confirmée (identique au document : Sergio lead tech, Dine UI/UX, Mickael data, Gaitant règles qualité, Omega compatibilité/impact).

H1:30 – H2 — Verrouillage du scénario

- Rédaction écrite du scénario exact (lots, quantités, trajets, score attendu) partagée à toute l'équipe. C'est la référence unique pendant les 36h.
-

PHASE 2 — H2 à H8 : Construction du cœur

- **Sergio** : modèle de données du lot, moteur de compatibilité, moteur de regroupement, modèle d'opération, modèle d'événement.
- **Dine** : layout général, sidebar, dashboard (structure sans design final), formulaire de création de lot, tableau des lots.
- **Mickael** : 10 à 15 lots fictifs réalistes, 5 localisations, 3 destinations, historique de transport en JSON.
- **Gaitant** : moteur de règles (retard, contrainte incompatible, seuils de vigilance).
- **Omega** : matrice de compatibilité pondérée (produit 30%, destination 25%, zone 20%, disponibilité 15%, contraintes 10%), logique de scoring.

Checkpoint H8 : on doit pouvoir créer un lot → le voir apparaître → lancer l'analyse → voir un score de compatibilité, même sans design.

PHASE 3 — H8 à H16 : Parcours complet

Construire dans l'ordre :

1. Dashboard (total lots, opportunités, opérations, alertes).
2. Page Matching (cartes de lots avec score de compatibilité affiché).
3. Page Grouping (proposition de regroupement, bouton "Créer l'opération").
4. Page Opération (timeline : lots enregistrés → regroupement validé → transport planifié → en transit → arrivé).
5. Passeport numérique (`/agroflow/passport/AF-001`) avec QR code.

Checkpoint H16 : le parcours complet doit fonctionner de bout en bout — Lot → Analyse → Matching → Grouping → Opération → Événement → Passeport. À ce stade, le projet est déjà pitchable même sans finition visuelle.

PHASE 4 — H16 à H22 : Polish + effets Wow

- Dine : espacement, typographie, cartes, états vides, animations légères, loading states.
- Trois effets Wow à intégrer si le temps le permet :

1. Score de compatibilité animé ("92% – HIGH COMPATIBILITY").
 2. Représentation simplifiée des flux (Bohicon → Allada → Abomey-Calavi → Cotonou).
 3. Passeport numérique scannable via QR code, avec statut "Traceability: Verified".
-

PHASE 5 — H22 à H26 : Stabilisation

Stop total aux nouvelles fonctionnalités. Uniquement du bug fixing.

Checklist :

- Tous les boutons fonctionnent.
 - Aucun écran vide, aucun "undefined".
 - Aucun crash.
 - Affichage correct sur l'écran de projection.
 - Données réalistes partout.
 - Navigation fluide.
 - Démo reproductible au moins 3 fois de suite.
-

PHASE 6 — H26 à H29 : Préparation du pitch

Structure recommandée (identique à la version 48h) :

1. Titre + accroche ("Mutualiser les volumes.
Organiser les flux. Suivre les lots.")
 2. Le problème (producteurs isolés, véhicules sous-remplis).
 3. L'opportunité (3 producteurs → 1 opération → 950 kg).
 4. AgroFlow : mutualisation, compatibilité, traçabilité.
 5. Démonstration live.
 6. Technologie (React, moteur de données, algorithme de compatibilité, passeport numérique, QR code) + mention de Cursor comme accélérateur de développement.
 7. Impact attendu (sans statistiques inventées).
 8. Vision (prototype → réseau logistique agricole → infrastructure régionale).
-

PHASE 7 — H29 à H36 : Répétitions + sécurisation

Créneau	Action
H29	Première répétition complète, chronométrée

Créneau	Action
H30	Corrections : passages trop longs, bugs, slides inutiles
H31	Deuxième répétition
H32	Simulation de questions du jury
H33 – H35	Ajustements finaux uniquement (pas de nouvelles features)
H35 – H36	Marge tampon : sauvegarde du projet, test sur la machine de présentation, vérification du mode hors-ligne

Cette marge tampon de fin (absente du planning 48h) est la principale différence : en 36h, il faut se réserver explicitement du temps pour l'imprévu technique de dernière minute.

3. Prompts Cursor optimisés pour économiser les tokens

Principes généraux à respecter dans chaque prompt

- Toujours préciser **le fichier exact** concerné (utiliser `@nom_du_fichier` dans Cursor) plutôt

que de coller tout le code dans le prompt.

- Demander des **diffs ciblés** ("modifie uniquement la fonction X") plutôt que "réécris le fichier".
- Découper les demandes en **petites unités fonctionnelles** (un composant, une fonction, un type) plutôt qu'une fonctionnalité entière en un seul prompt.
- Donner le **contexte minimal mais suffisant** : structure de données attendue, format d'entrée/sortie, contraintes — pas l'historique complet du projet.
- Éviter les prompts ouverts type "améliore le code" : préférer des critères précis ("réduis la complexité", "extraie cette logique dans un hook").
- Réutiliser un même fichier de contexte (ex. `types/lot.ts`) comme référence commune pour éviter de le redécrire à chaque prompt.

Prompt — Initialisation du projet (Sergio)

```
Crée la structure de dossiers suivante dans
un projet Vite + React + Tailwind :
src/components, src/pages, src/data,
src/services, src/utils, src/types.
N'installe et ne configure que react-
router-dom, lucide-react, qrcode.react,
recharts.
Ne génère aucun composant pour l'instant,
```

seulement la structure et la config de base.

Prompt – Modèle de données du lot

Dans `src/types/lot.ts`, définis un type TypeScript "Lot" avec les champs :
id, produit, quantiteKg, localisation, destination, dateDisponibilite, contraintes (string[]), statut.
Ajoute aussi un type "Operation" regroupant plusieurs lots avec un score de compatibilité.
Ne génère aucune logique métier, uniquement les types.

Prompt – Moteur de compatibilité

Dans `src/services/compatibility.ts`, écris une fonction
`calculateCompatibilityScore(lotA, lotB)`
qui retourne un score sur 100 selon cette pondération :
produit/catégorie 30%, destination 25%, zone géographique 20%, disponibilité 15%, contraintes 10%.
Utilise les types définis dans `src/types/lot.ts` (référence-les, ne les redéfinis pas).
Retourne uniquement le code de la fonction, pas d'explication.

Prompt — Composant UI (Dine)

Dans `src/components/LotCard.tsx`, crée un composant React qui affiche une carte de lot avec : produit, quantité, trajet (localisation → destination), et une barre de score de compatibilité. Utilise uniquement Tailwind pour le style, pas de CSS séparé. Le composant prend un objet "Lot" en prop (type importé depuis `src/types/lot.ts`).

Prompt — Génération de données de test (Mickael)

Dans `src/data/lots.json`, génère 12 lots agricoles fictifs réalistes au format du type Lot (produits variés : maïs, ananas, soja ; localisations au Bénin ; 3 destinations communes).
Retourne uniquement le JSON, sans commentaire.

Prompt — Moteur de règles / alertes (Gaitant)

Dans `src/services/rulesEngine.ts`, écris une fonction `checkAlerts(operation)` qui retourne

```
une liste d'alertes selon ces règles :  
- retard > 24h → "vigilance élevée"  
- contrainte produit incompatible entre  
deux lots → "regroupement refusé"  
Utilise le type Operation existant. Code  
uniquement, sans explication.
```

Prompt – Debug ciblé (à utiliser en phase de stabilisation)

```
Le composant @OperationTimeline.tsx affiche  
"undefined" au lieu du statut quand  
operation.status  
est vide. Corrige uniquement ce cas, sans  
modifier le reste du fichier.
```

Prompt – Passeport numérique + QR code

```
Dans src/pages/Passport.tsx, crée une page  
qui affiche les informations d'un lot  
(id, produit, producteur, origine,  
destination, statut, historique) à partir  
de son id  
passé en paramètre d'URL. Ajoute un QR code  
(qrcode.react) qui encode l'URL de la page.  
Style Tailwind sobre, pas d'animation.
```

Astuces supplémentaires pour limiter la consommation de tokens

- Fermer les onglets/fichiers non pertinents dans Cursor avant de lancer un prompt : Cursor inclut souvent le contexte des fichiers ouverts.
- Préférer plusieurs prompts courts et séquentiels à un seul prompt long qui demande "toute la page" — cela réduit aussi le risque d'erreurs à corriger ensuite.
- Pour la génération de données JSON volumineuses, demander un petit échantillon (2-3 lots) puis dupliquer/adapter manuellement plutôt que de faire générer 15 lots en un seul prompt.
- Réserver les prompts "explique-moi" ou "documente ce code" pour la toute fin (phase pitch), pas pendant le sprint de développement.